



영화 리뷰 & 감성 분석 웹 애플리케이션

Streamlit 프론트엔드 + FastAPI 백엔드

리뷰 감성 자동 분석 서비스

미션	스프린트 미션 18 (최종)
프론트엔드	Streamlit
백엔드	FastAPI
데이터베이스	SQLite (표준 라이브러리 sqlite3)
감성 분석	KcELECTRA 계열 + 규칙 기반 풀백

GitHub	https://github.com/minjaejeon0827/sprint_mission18
Docker Hub (프론트엔드)	https://hub.docker.com/r/minjaejeon0827/frontend-movie-review
Docker Hub (백엔드)	https://hub.docker.com/r/minjaejeon0827/backend-movie-review

Streamlit

FastAPI

SQLite

Transformers

Docker

1 서비스 개요

본 서비스는 **영화 정보를 관리하고, 사용자 리뷰를 등록하며, 등록된 리뷰의 감성(긍정·부정·중립)을 자동으로 분석해 보여주는 웹 애플리케이션**입니다. 사용자는 웹 화면에서 영화를 목록으로 둘러보고, 새 영화를 추가하고, 각 영화에 리뷰를 남길 수 있습니다. 리뷰가 등록되는 순간 백엔드가 그 내용을 감성 분석 모델에 통과시켜 긍정·부정·중립 중 하나로 분류하고, 그 결과를 함께 저장합니다.

전체 구조는 **화면(프론트엔드)과 데이터 처리 로직(백엔드)을 분리**하는 표준적인 웹 아키텍처를 따릅니다. 화면은 파이썬만으로 빠르게 인터랙티브 UI를 만들 수 있는 **Streamlit**으로 구현했고, 데이터의 검증·저장·조회와 감성 분석 모델 서빙은 고성능 API 서버인 **FastAPI**가 담당합니다. 두 구성 요소는 HTTP(REST API)로 통신하며, 이 분리 구조 덕분에 같은 백엔드 API를 모바일 앱이나 다른 서비스에서도 재사용할 수 있습니다.

핵심 기능

- **영화 관리** — 영화 등록(제목·개봉일·감독·장르·포스터 URL), 전체/특정 영화 조회, 특정 영화 삭제
- **리뷰 관리** — 리뷰 등록, 특정 영화의 리뷰 조회, 전체 리뷰(최근순) 조회, 리뷰 삭제
- **리뷰 감성 분석** — 리뷰 등록 시 자동으로 긍정·부정·중립을 판정하고, 긍정 확률 점수를 함께 저장
- **평점(평균 감성 점수) 조회** — 한 영화에 달린 모든 리뷰의 감성 점수 평균을 계산해 제공
- **API 키 인증** — 헤더의 API 키를 검증해 허가된 요청만 처리(**X-API-Key**)

사용 기술 스택

구분	기술	역할
프론트엔드	Streamlit	사용자 화면(영화 목록·추가·리뷰) 구성 및 API 호출
백엔드	FastAPI + Uvicorn	REST API 서버, 자동 문서화(/docs), 입력 검증
데이터베이스	SQLite (<code>sqlite3</code>)	영화·리뷰 데이터의 영구 저장 (표준 라이브러리만 사용)
감성 분석 모델	Transformers (KcELECTRA 계열)	한국어 리뷰의 긍정·부정·중립 분류
배포	Docker / Streamlit Cloud / GCP Cloud Run	컨테이너 기반 배포 환경

설계 원칙 — 본 프로젝트는 파이썬 표준 라이브러리 `sqlite3` 만으로 구현했습니다. `sqlite3.connect` 로 커넥션을 만들고, 파라미터화된 SQL 쿼리(`?` 플레이스홀더)로 직접 CRUD 수행하여 SQL 인젝션을 방지합니다.

2 서비스 구조도

서비스는 네 개의 계층으로 구성됩니다. 사용자와 직접 상호작용하는 **프론트엔드(Streamlit)**, 요청을 검증·처리하는 **백엔드(FastAPI)**, 데이터를 영구 저장하는 **데이터베이스(SQLite)**, 그리고 리뷰 텍스트의 감성을 분석하는 **모델 서빙 계층**입니다. 각 계층 간의 데이터 흐름은 아래 그림과 같습니다.

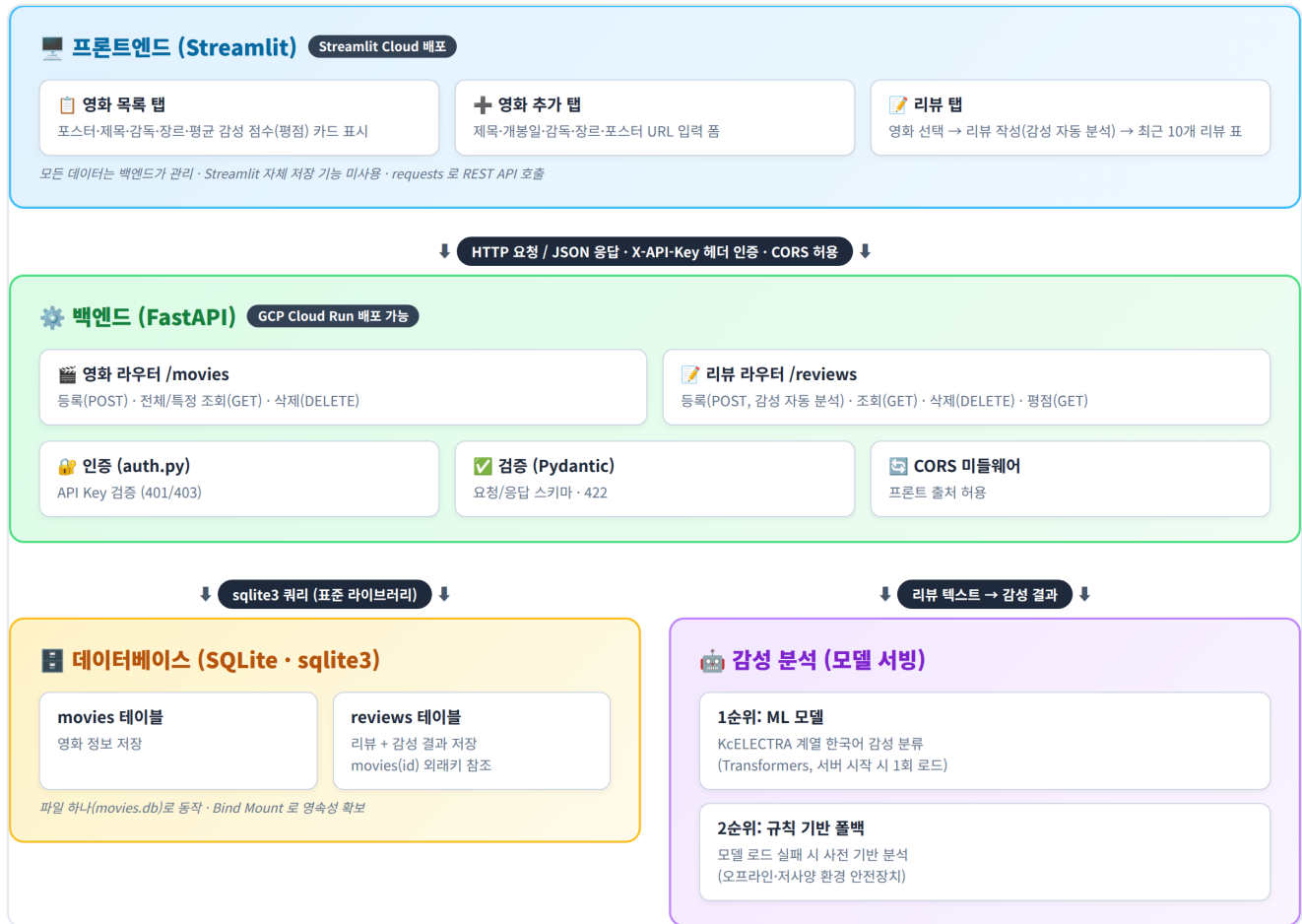


그림 2-1. 전체 서비스 아키텍처 (프론트엔드 · 백엔드 · 데이터베이스 · 모델 서빙)

계층별 상세

① 프론트엔드 (Streamlit)

세 개의 탭으로 구성됩니다. **영화 목록** 탭은 등록된 영화를 포스터·제목·감독·장르·평균 감성 점수와 함께 카드 형태로 보여줍니다. **영화 추가** 탭은 제목·개봉일·감독·장르·포스터 URL을 입력받아 등록합니다. **리뷰** 탭은 영화를 선택

해 리뷰를 작성하면 감성 분석이 자동 실행되고, 화면 하단에 최근 10개 리뷰를 표로 보여줍니다. Streamlit은 자체적으로 데이터를 저장하지 않으며, 모든 데이터는 `requests` 라이브러리를 통해 백엔드 API에서 주고받습니다.

② 백엔드 (FastAPI)

영화 라우터(`/movies`)와 리뷰 라우터(`/movies/{id}/reviews` , `/reviews`)로 기능을 분리했습니다. 모든 요청은 **Pydantic 스키마로 검증**되어 잘못된 입력은 자동으로 422 오류로 걸러지고, **API 키 인증**을 통과한 요청만 처리됩니다. **CORS 미들웨어**를 설정해 다른 출처(포트)에서 실행되는 Streamlit 화면이 API를 정상 호출할 수 있도록 허용합니다.

③ 데이터베이스 (SQLite · sqlite3)

`movies` 와 `reviews` 두 개의 테이블로 구성됩니다. `reviews.movie_id` 가 `movies.id` 를 참조하는 외래 키 관계이며, **ON DELETE CASCADE** 설정으로 영화를 삭제하면 그 영화의 리뷰도 함께 삭제됩니다. 파일 하나 (`movies.db`)로 동작하므로 별도의 DB 서버 설치가 필요 없고, Docker의 Bind Mount와 함께 사용하면 컨테이너를 삭제해도 데이터가 보존됩니다.

④ 모델 서빙 (감성 분석)

리뷰 텍스트의 감성을 **2단계 풀백 구조**로 분석합니다. 1순위는 Hugging Face의 한국어 감성 분류 모델 (KcELECTRA 계열)로, 서버 시작 시 `lifespan` 에서 한 번만 로드해 메모리에 상주시킵니다. 만약 오프라인 환경이나 저사양 서버 등으로 모델 로드 실패하면, 자동으로 **규칙 기반(사전 기반) 분석**으로 전환되어 서비스가 멈추지 않도록 안전장치를 두었습니다. 이 경량화 전략은 5장에서 자세히 설명합니다.

3 데이터베이스 구조도 (ERD)

데이터베이스는 **영화(movies)**와 **리뷰(reviews)** 두 엔티티로 구성되며, 두 엔티티는 **1 대 N 관계**입니다. 즉 하나의 영화에 여러 개의 리뷰가 달릴 수 있으며, 각 리뷰는 반드시 하나의 영화에 속합니다.



그림 3-1. 엔티티 관계도(ERD) — movies(1) : reviews(N)

movies 테이블

컬럼명	타입	제약	설명
id	INTEGER	PK · AUTOINCREMENT	영화 고유 번호 (자동 생성)
title	TEXT	NOT NULL	영화 제목
release_date	TEXT	-	개봉일 (예: 2014-11-06)
director	TEXT	-	감독
genre	TEXT	-	장르
poster_url	TEXT	-	포스터 이미지 URL
created_at	TEXT	NOT NULL	등록 일시 (기본값: 현재 시각)

reviews 테이블

컬럼명	타입	제약	설명
id	INTEGER	PK • AUTOINCREMENT	리뷰 고유 번호 (자동 생성)
movie_id	INTEGER	FK → movies.id NN	이 리뷰가 속한 영화 번호
author	TEXT	NOT NULL	작성자 이름
content	TEXT	NOT NULL	리뷰 내용
sentiment	TEXT	-	감성 분석 결과 (긍정/부정/중립)
score	REAL	-	긍정 확률 점수 (0.0 ~ 1.0)
created_at	TEXT	NOT NULL	등록 일시 (기본값: 현재 시각)

외래키와 CASCADE — `reviews.movie_id` 는 `movies.id` 를 참조하는 외래키입니다. 테이블 정의에 `ON DELETE CASCADE` 를 지정하고, 커넥션마다 `PRAGMA foreign_keys = ON` 을 실행하여 외래키 제약이 실제로 강제되도록 했습니다. (SQLite는 기본적으로 외래키 검사가 꺼져 있습니다.) 이 덕분에 영화를 삭제하면 그 영화에 달린 리뷰가 자동으로 함께 삭제되어 고아 데이터가 남지 않습니다.

4 API 엔드포인트 명세

백엔드는 총 11개의 엔드포인트를 제공합니다. 영화·리뷰에 대한 CRUD와 평점 조회, 그리고 루트·헬스 체크로 구성되어 있으며, 루트(/)와 헬스 체크(/health)를 제외한 모든 엔드포인트는 `X-API-Key` 헤더 인증을 요구합니다.

메서드	경로	기능
POST	/movies	영화 등록
GET	/movies	전체 영화 조회 (평균 감성 점수 포함)
GET	/movies/{movie_id}	특정 영화 조회
DELETE	/movies/{movie_id}	특정 영화 삭제 (리뷰 함께 삭제)
POST	/movies/{movie_id}/reviews	리뷰 등록 (감성 자동 분석)
GET	/movies/{movie_id}/reviews	특정 영화의 리뷰 조회
GET	/reviews	전체 리뷰 조회 (최근순, 기본 10개)
GET	/movies/{movie_id}/rating	특정 영화의 평점(감성 점수 평균)
DELETE	/reviews/{review_id}	특정 리뷰 삭제
GET	/	루트 (서비스 동작 확인)
GET	/health	헬스 체크 (로드밸런서·모니터링용)

FastAPI 자동 문서 (Swagger UI)

FastAPI는 코드에 작성한 타입 힌트와 설명(docstring)을 바탕으로 `/docs` 경로에 대화형 API 문서(Swagger UI)를 자동 생성합니다. 아래는 실제 배포된 서버(`movie-review-api-service...run.app`)의 `/docs` 화면을 엔드포인트별로 캡처한 것이며, 각 엔드포인트의 설명·파라미터·요청 바디·응답 형식이 명세와 함께 표시됩니다. 실제로 **Try it out**으로 호출한 응답(Response body)까지 함께 담았습니다.

movies

GET

/movies

Get Movies

전체 영화 목록을 조회합니다. 각 영화에 대해 리뷰 개수와 감성 점수 평균(평점)을 함께 계산해 붙입니다.

LEFT JOIN 을 사용하므로, 리뷰가 하나도 없는 영화도 목록에서 빠지지 않습니다. (이 경우 avg_score 는 NULL, review_count 는 0 이 됩니다.)

Parameters

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies' \
  -H 'accept: application/json' \
  -H 'X-API-Key: my-secret-key-12345'
```

Request URL

https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies

Server response

Code

Details

200

Response body

```
[
  {
    "id": 9,
    "title": "올드보이",
    "release_date": "2003-11-21",
    "director": "박찬욱",
    "genre": "스릴러, 미스터리",
    "poster_url": "https://i.namu.wiki/i/0CEqc_oqX8xtL_JHjupyFHQ1Vhkp-S__1nd237zbckK0X-idXwZYVwKaGKtsLR-orkmvpZor40z08_B_qvoYGN0NBiDk17z7xck1cn60xd0LBW0Pg9HS_qQPz0T1FMsAGVtZe10pv03p5p0m8sYkg.webp",
    "created_at": "2026-07-30 01:38:12",
    "avg_score": 0.0836,
    "review_count": 1
  },
  {
    "id": 4,
    "title": "라라랜드",
    "release_date": "2016-12-07",
    "director": "데이미언 셔젤",
    "genre": "뮤지컬, 로맨스",
    "poster_url": "https://i.namu.wiki/i/noiIu0Zpk7gSNvcWR4PjHu4d-6yv30UQW0NvDc58mxdxXADmytQBLP0hd8qVNYwFNpV8Bnsh0zDmpZPPcI4IeZyIAPV11EtPycws6Fw7F0_Dc9FXqS_VJzPEu8u3vG7kpbTo061tGy0nzEK0feI9dA.webp",
    "created_at": "2026-07-29 23:44:20",
    "avg_score": 0.9706,
    "review_count": 1
  },
  {
    "id": 3,
```

그림 4-1. GET /movies — 전체 영화 목록 조회 (평균 감성 점수 포함)

10 / 28

POST

/movies Create Movie

영화를 등록합니다. 파라미터화된 쿼리(?, ?)를 사용해 SQL 인젝션을 방지합니다.

Parameters

Try it out

Reset

No parameters

Request body required

application/json

Example Value

Schema

```

{
  "director": "장항준",
  "genre": "사극, 드라마",
  "poster_url": "https://i.namu.wiki/i/LYEG3-94SnSKil7NJeR-z-qJv7LYQ4m6dnMaEiltcFEPmXgT7ZCvr2v9HmuchZtI0aFdckBoxgZtKS9F0akmUgeVoJ2EB69hp90LyMkEoyo5Cyr-BUIwzvicQCocdI3n1IC_YQ0qkvA18HF9dVC8qQ.webp",
  "release_date": "2026-02-04",
  "title": "왕과 사는 남자"
}

```

Responses

Curl

```

curl -X 'POST' \
  'https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies' \
  -H 'accept: application/json' \
  -H 'X-API-Key: my-secret-key-12345' \
  -H 'Content-Type: application/json' \
  -d '{
    "director": "장항준",
    "genre": "사극, 드라마",
    "poster_url": "https://i.namu.wiki/i/LYEG3-94SnSKil7NJeR-z-qJv7LYQ4m6dnMaEiltcFEPmXgT7ZCvr2v9HmuchZtI0aFdckBoxgZtKS9F0akmUgeVoJ2EB69hp90LyMkEoyo5Cyr-BUIwzvicQCocdI3n1IC_YQ0qkvA18HF9dVC8qQ.webp",
    "release_date": "2026-02-04",
    "title": "왕과 사는 남자"
  }'

```

Request URL

```
https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies
```

Server response

Code

Details

201

Response body

```

{
  "id": 10,
  "title": "왕과 사는 남자",
  "release_date": "2026-02-04",
  "director": "장항준",
  "genre": "사극, 드라마",
  "poster_url": "https://i.namu.wiki/i/LYEG3-94SnSKil7NJeR-z-qJv7LYQ4m6dnMaEiltcFEPmXgT7ZCvr2v9HmuchZtI0aFdckBoxgZtKS9F0akmUgeVoJ2EB69hp90LyMkEoyo5Cyr-BUIwzvicQCocdI3n1IC_YQ0qkvA18HF9dVC8qQ.webp",
  "created_at": "2026-07-30 02:20:11",
  "avg_score": null,
  "review_count": 0
}

```

Download

그림 4-2. POST /movies — 영화 등록 (요청 바디 · 201 응답)

GET /movies/{movie_id} Get Movie

특정 영화를 조회합니다. 없으면 404.

Parameters

Cancel

Name	Description
movie_id * required	
integer (path)	5

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies/5' \
-H 'accept: application/json' \
-H 'X-API-Key: my-secret-key-12345'
```

Request URL

https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies/5

Server response

Code	Details
404 Undocumented	Error: response status is 404 Response body <pre>{ "detail": "해당 영화를 찾을 수 없습니다." }</pre> Download Response headers <pre>alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000 content-length: 54 content-type: application/json date: Thu, 30 Jul 2026 02:23:20 GMT server: Google Frontend x-cloud-trace-context: a32f6a4e456b674f19ce2a27d0a68208</pre>

Responses

Code	Description	Links
------	-------------	-------

그림 4-3. GET /movies/{movie_id} — 특정 영화 조회 (없으면 404)

DELETE

/movies/{movie_id} Delete Movie



특정 영화를 삭제합니다. 외래키 ON DELETE CASCADE 설정 덕분에, 이 영화에 달린 리뷰도 함께 삭제됩니다.

Parameters

Cancel

Name	Description
movie_id * required integer (path)	10

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
'https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies/10' \
-H 'accept: application/json' \
-H 'X-API-Key: my-secret-key-12345'
```

Request URL

```
https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies/10
```

Server response

Code	Details
200	Response body <pre>{ "message": "'월과 사는 남자' 영화가 삭제되었습니다." }</pre>  Download Response headers <pre>access-control-allow-credentials: true alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000 content-length: 69 content-type: application/json date: Thu,30 Jul 2026 02:25:30 GMT server: Google Frontend x-cloud-trace-context: 5d0c33e4bedf611faea48ad33633cac4;o=1</pre>

Responses

Code	Description	Links
------	-------------	-------

그림 4-4. DELETE /movies/{movie_id} — 특정 영화 삭제 (리뷰 CASCADE)

13 / 28

reviews

POST

/movies/{movie_id}/reviews

Create Review

리뷰를 등록합니다. 등록 직전에 감성 분석(analyze_sentiment)을 실행해 그 결과(sentiment, score)를 리뷰와 함께 저장합니다.

Parameters

Try it out

Reset

Name

Description

movie_id * required

integer

(path)

9

Request body required

application/json

Example Value | Schema

{

"author": "agai****",

"content": "몰래함의 찌꺼기가 남는건 어쩔 수 없구나"

}

Responses

Curl

curl -X 'POST' \

'https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies/9/reviews' \

-H 'accept: application/json' \

-H 'X-API-Key: my-secret-key-12345' \

-H 'Content-Type: application/json' \

-d '{

"author": "agai****",

"content": "몰래함의 찌꺼기가 남는건 어쩔 수 없구나"

}'

Request URL

https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies/9/reviews

Server response

Code

Details

201

Response body

{

"id": 7,

"movie_id": 9,

"author": "agai****",

"content": "몰래함의 찌꺼기가 남는건 어쩔 수 없구나",

"sentiment": "부정",

"score": 0.0836,

"created_at": "2026-07-30 02:28:50"

}

Download

그림 4-5. POST /movies/{movie_id}/reviews — 리뷰 등록 (감성 자동 분석)

GET

/movies/{movie_id}/reviews

Get Reviews By Movie

특정 영화에 달린 리뷰를 최신순으로 조회합니다.

Parameters

Try it out

Name	Description
movie_id * required integer (path)	9

Responses

Curl

```
curl -X 'GET' \
  'https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies/9/reviews' \
  -H 'accept: application/json' \
  -H 'X-API-Key: my-secret-key-12345'
```

Request URL

https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies/9/reviews

Server response

Code	Details
200	Response body <pre>[{ "id": 8, "movie_id": 9, "author": "kdw0****", "content": "세치화로 자은 죄의 무게를", "sentiment": "부정", "score": 0.8749, "created_at": "2026-07-30 02:30:59" }, { "id": 7, "movie_id": 9, "author": "agai****", "content": "불레함의 피꺼기가 남는건 어쩔 수 없구나", "sentiment": "부정", "score": 0.8836, "created_at": "2026-07-30 02:28:50" }]</pre> Download

Response headers

```
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
content-length: 345
content-type: application/json
date: Thu, 30 Jul 2026 02:31:42 GMT
server: Google Frontend
x-cloud-trace-context: 0035230b328275ca9a41e3e9f6498b6b;o=1
```

그림 4-6. GET /movies/{movie_id}/reviews — 특정 영화의 리뷰 조회

GET

/reviews Get All Reviews

🔒 ⤴

전체 리뷰를 최신순으로 조회합니다. 기본값 limit=10 → 프론트엔드의 '최근 10개 리뷰' 요구사항에 대응합니다.

Parameters

Cancel

Name	Description
limit integer (query)	가져올 리뷰 개수 2 maximum: 100 minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'https://movie-review-api-service-778186500886.asia-northeast3.run.app/reviews?limit=2' \
-H 'accept: application/json' \
-H 'X-API-Key: my-secret-key-12345'
```

📄 📄

Request URL

```
https://movie-review-api-service-778186500886.asia-northeast3.run.app/reviews?limit=2
```

Server response

Code	Details
200	Response body <pre>[{ "id": 8, "movie_id": 9, "author": "kdw*****", "content": "새치허브 지은 죄의 무게를", "sentiment": "부정", "score": 0.8749, "created_at": "2026-07-30 02:30:59" }, { "id": 7, "movie_id": 9, "author": "agai*****", "content": "블레함의 피짜가가 남는건 어쩔 수 없구나", "sentiment": "부정", "score": 0.8836, "created_at": "2026-07-30 02:28:50" }]</pre> 📄 Download

전체 리뷰를 최신순으로 조회합니다. 기본값 limit=10 → 프론트엔드의 '최근 10개 리뷰' 요구사항에 대응합니다.

Parameters

Cancel

Name	Description
limit integer (query)	가져올 리뷰 개수 2 maximum: 100 minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'https://movie-review-api-service-778186500886.asia-northeast3.run.app/reviews?limit=2' \
-H 'accept: application/json' \
-H 'X-API-Key: my-secret-key-12345'
```

Request URL

<https://movie-review-api-service-778186500886.asia-northeast3.run.app/reviews?limit=2>

Server response

Code

Details

200

Response body

```
{
  {
    "id": 8,
    "movie_id": 9,
    "author": "kdog****",
    "content": "새치러로 지은 죄의 무게음",
    "sentiment": "부정",
    "score": 0.6749,
    "created_at": "2026-07-30 02:30:59"
  },
  {
    "id": 7,
    "movie_id": 9,
    "author": "agai****",
    "content": "물때함의 피꺼기가 남는건 어쩔 수 없구나",
    "sentiment": "부정",
    "score": 0.6836,
    "created_at": "2026-07-30 02:28:50"
  }
}
```

Download

그림 4-7. GET /reviews — 전체 리뷰 조회 (최근순, limit)

GET

/movies/{movie_id}/rating

Get Movie Rating

특정 영화의 평점을 조회합니다. 평점 = 그 영화에 달린 모든 리뷰의 감성 점수(score, 긍정 확률) 평균. 리뷰가 없으면 avg_score 는 None, review_count 는 0 입니다.

Parameters

Cancel

Name	Description
movie_id ★ required integer (path)	9

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies/9/rating' \
  -H 'accept: application/json' \
  -H 'X-API-Key: my-secret-key-12345'
```

Request URL

https://movie-review-api-service-778186500886.asia-northeast3.run.app/movies/9/rating

Server response

Code	Details
200	Response body <pre>{ "movie_id": 9, "avg_score": 0.0792, "review_count": 2 }</pre> Download

Response headers

```
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
content-length: 50
content-type: application/json
date: Thu, 30 Jul 2026 02:35:21 GMT
server: Google Frontend
x-cloud-trace-context: 850276c63b026b7ea3c04b3e1d421652;o=1
```

그림 4-8. GET /movies/{movie_id}/rating — 평점(감성 점수 평균) 조회

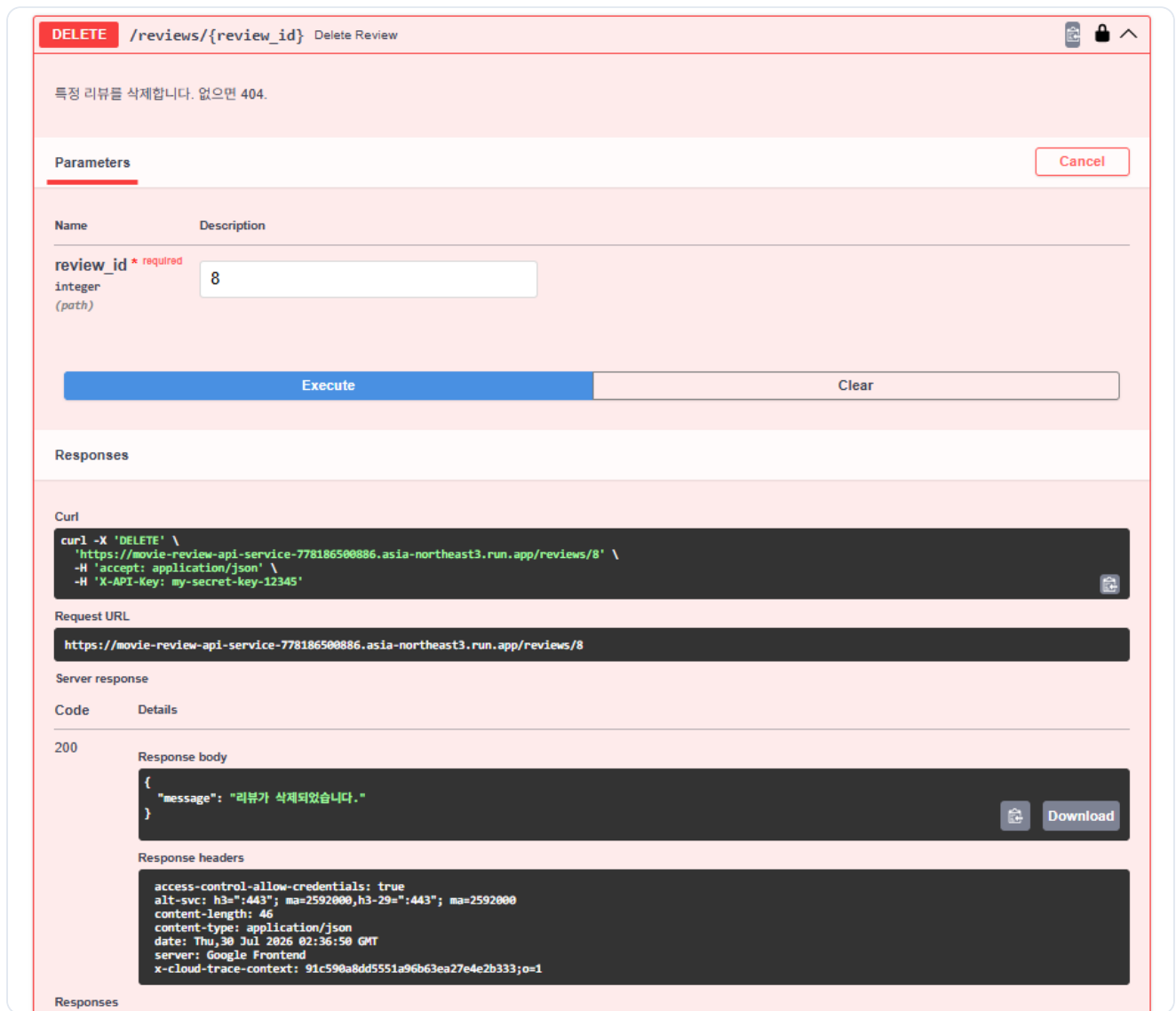


그림 4-9. DELETE /reviews/{review_id} — 특정 리뷰 삭제

default

GET / Read Root

루트 — 서비스가 살아있는지 간단히 확인.

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://movie-review-api-service-778186500886.asia-northeast3.run.app/' \
  -H 'accept: application/json'
```

Request URL

```
https://movie-review-api-service-778186500886.asia-northeast3.run.app/
```

Server response

Code	Details
200	Response body <pre>{ "message": "영화 리뷰 & 감상 분석 API 입니다. /docs 에서 문서를 확인하세요." }</pre> Download Response headers <pre>alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000 content-length: 98 content-type: application/json date: Thu,30 Jul 2026 02:38:06 GMT server: Google Frontend x-cloud-trace-context: 40d0079ad7cca9c119f9781c3499e0;o=1</pre>

Responses

그림 4-10. GET / — 루트 (서비스 동작 확인)

19 / 28

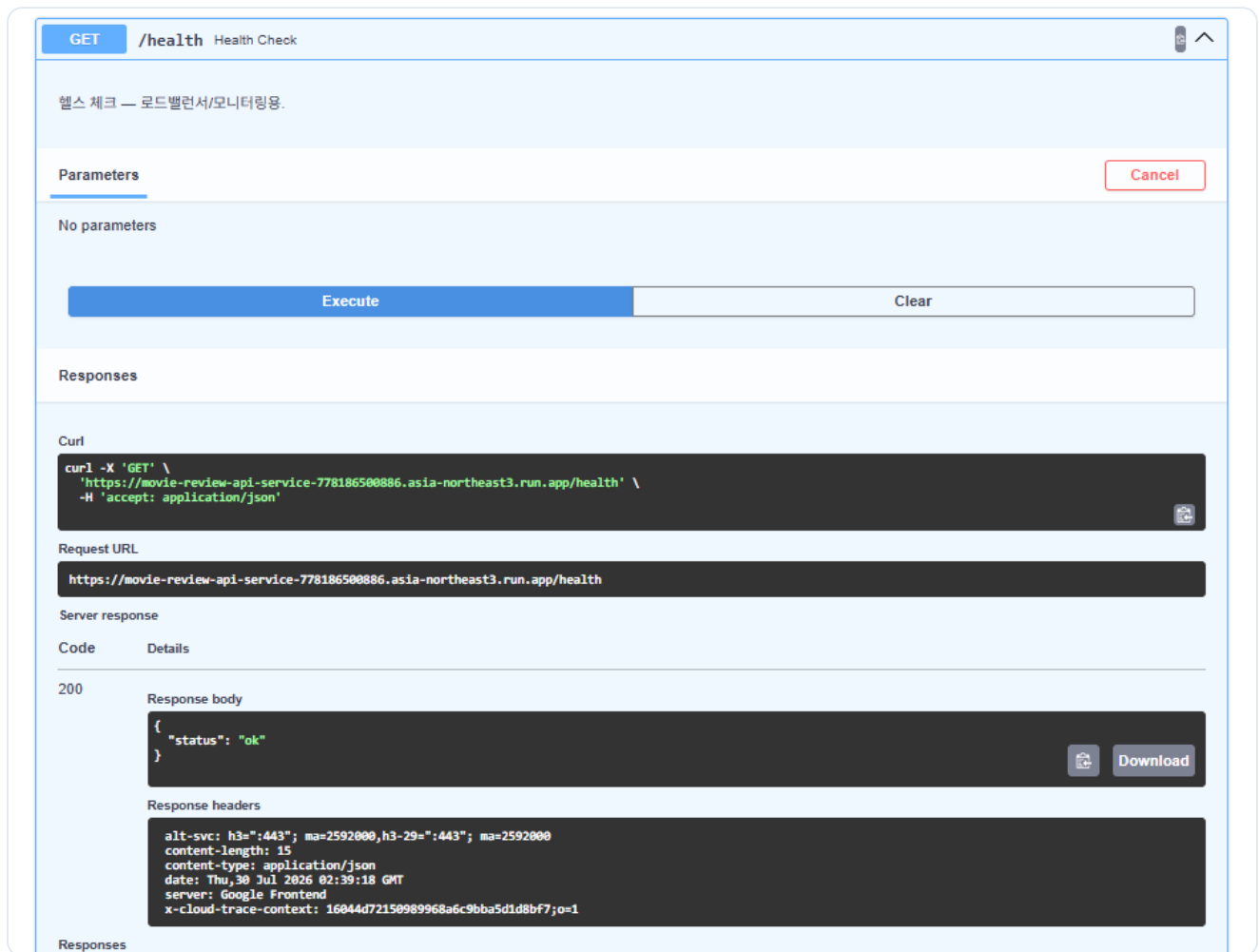


그림 4-11. GET /health — 헬스 체크 (로드밸런서·모니터링용)

Schemas	
HTTPValidationError ^ Collapse all object detail > Expand all array<object>	
MovieCreate ^ Collapse all object 영화 등록 요청 바디 title* > Expand all string [1, 200] characters release_date > Expand all (string null) director > Expand all (string null) genre > Expand all (string null) poster_url > Expand all (string null) Example > Expand all object	
MovieResponse ^ Collapse all object 영화 응답 형태 id* integer title* string release_date > Expand all (string null) director > Expand all (string null) genre > Expand all (string null) poster_url > Expand all (string null) created_at > Expand all (string null) avg_score > Expand all (number null) review_count > Expand all integer	
RatingResponse ^ Collapse all object 특정 영화의 평점(감성 점수 평균) 응답 movie_id* integer avg_score > Expand all (number null) review_count > Expand all integer	
ReviewCreate ^ Collapse all object 리뷰 등록 요청 바디 author* > Expand all string [1, 100] characters content* > Expand all string [1, 2000] characters Example > Expand all object	
ReviewResponse ^ Collapse all object 리뷰 응답 형태 (감성 분석 결과 포함) id* integer movie_id* integer author* string content* string sentiment > Expand all (string null) score > Expand all (number null) created_at > Expand all (string null)	

그림 4-12. Schemas — 요청·응답 데이터 스키마 (MovieCreate, ReviewResponse 등)

5 리뷰 감성 분석 및 모델 경량화

모델 선정

리뷰는 영화에 대한 짧은 구어체 문장이 많고, 신조어나 오타자가 섞이는 특성이 있습니다. 이런 **사용자 생성 텍스트(User-Generated Text)**에는 잘 정제된 위키·뉴스로 학습한 일반 모델보다, 네이버 뉴스 댓글 등 실제 구어체 데이터로 학습한 **KcELECTRA 계열 모델**이 더 적합합니다. 본 프로젝트는 기본적으로 KcELECTRA 기반의 한국어 감성 분류 모델(`matthewburke/korean_sentiment`)을 사용하며, 환경변수 `SENTIMENT_MODEL` 로 다른 모델로 손쉽게 교체할 수 있도록 설계했습니다.

감성 판정 방식 — 모델이 출력한 라벨과 확신도를 항상 "**긍정 확률(0~1)**"로 정규화한 뒤, 0.6 이상이면 **긍정**, 0.4 이하이면 **부정**, 그 사이면 **중립**으로 3분류합니다. 이 긍정 확률을 `score` 컬럼에 저장하고, 한 영화의 평균 `score` 를 평점으로 활용합니다.

모델 경량화 전략

감성 분석 모델은 일반적으로 무겁기 때문에, 실서비스에서 효율적으로 운영하려면 경량화가 중요합니다. 본 프로젝트에 실제로 적용한 경량화·최적화 전략은 다음과 같습니다.

- **모델 1회 로드(싱글턴)** — 서버 시작 시 `lifespan` 에서 모델을 한 번만 로드해 메모리에 상주시키고, 이후 모든 요청이 이를 재사용합니다. 요청마다 재로드하는 낭비를 원천 제거합니다.
- **경량 아키텍처 채택** — BERT-base(약 1.1억 파라미터) 대신 파라미터 수가 더 적은 ELECTRA 계열을 사용해, GPU 없이 CPU만으로도 실용적인 추론 속도를 확보합니다.
- **추론 시 그래디언트 비활성화** — 추론 파이프라인은 학습이 필요 없으므로 그래디언트 계산을 끄고 (`torch.no_grad` 상당), 메모리와 속도를 개선합니다.
- **입력 길이 제한** — `max_length` 로 입력을 자름(truncation)으로써 아주 긴 리뷰가 들어와도 추론 시간이 안정적으로 유지됩니다.
- **CPU 스레드 수 조정** — CPU 코어의 절반만 사용하도록 제한해, 단일 서버에서 감성 분석이 다른 요청 처리를 과도하게 방해하지 않도록 합니다.

더 나아가 실제 프로덕션에서는 **동적 양자화(Dynamic Quantization)**로 가중치를 8비트로 줄이거나, **ONNX Runtime**으로 변환해 추론을 가속하거나, 여러 리뷰를 한꺼번에 처리하는 **배치 추론**을 도입하는 방향으로 추가 경량화가 가능합니다.

안정성을 위한 폴백(Fallback) — 모델 다운로드가 불가능한 오프라인 환경이나 메모리가 부족한 서버에서는 모델 로드가 실패할 수 있습니다. 이때 서비스가 멈추지 않도록, 긍정·부정 단어 사전을 기반으로 한 **규칙 기반 분석기**가 자동으로 대신 동작합니다. 이 이중 구조 덕분에 어떤 환경에서도 리뷰 감성 분석 기능이 항상 제공됩니다.

6 서비스 동작 화면

아래는 실제로 **Streamlit Cloud**에 배포된 **프론트엔드**(`codeit-movie-review.streamlit.app`)와 **GCP Cloud Run**에 배포된 **백엔드**를 연동해 동작시킨 화면입니다. 영화 5편(올드보이·라라랜드·어벤져스: 엔드게임·기생충·인터스텔라)을 등록하고, 각 영화에 리뷰를 등록했습니다. (요구사항: 영화 3개 이상, 각 영화당 리뷰 10개 이상 등록)

영화 목록 화면

등록된 영화가 포스터·제목·개봉일·감독·장르와 함께 카드로 표시되며, 각 카드에는 그 영화의 리뷰들로부터 계산된 **평균 감성 점수(평점)**가 함께 나타납니다. 왼쪽 사이드바의 **화면 설정** 슬라이더로 포스터 크기와 한 줄에 표시할 영화 수를 조절할 수 있습니다.

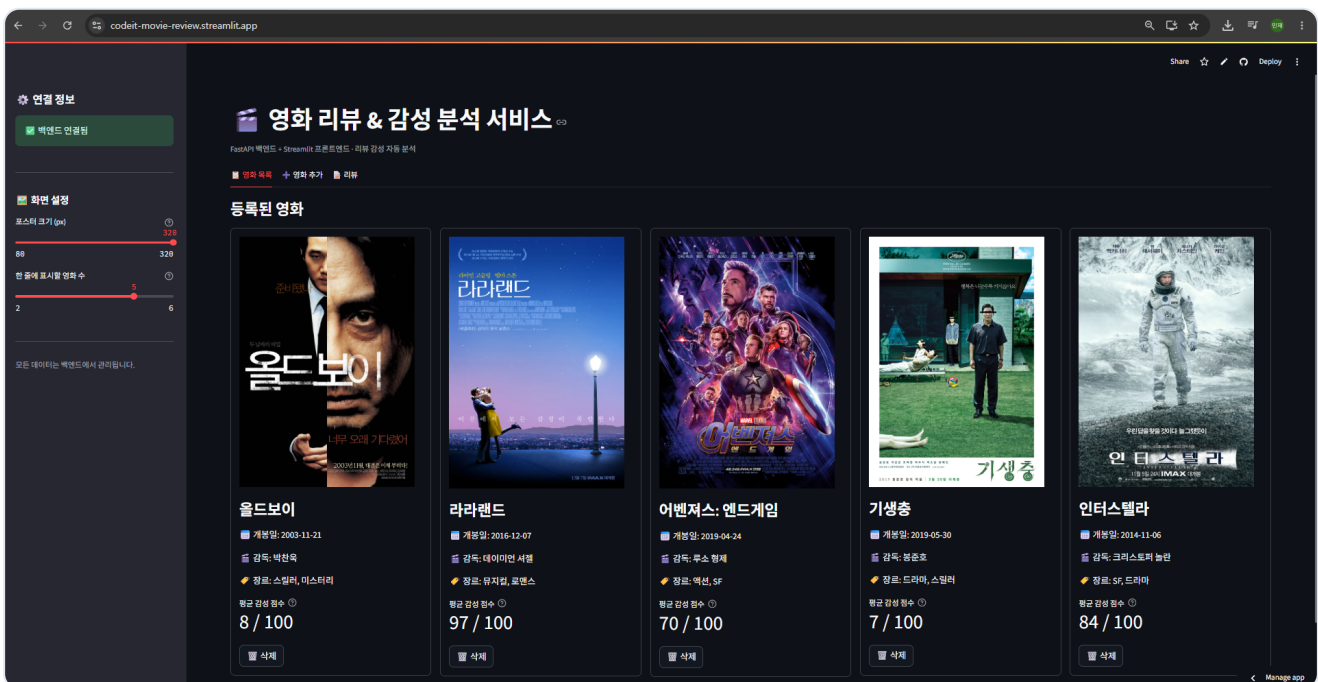


그림 6-1. 영화 목록 화면 — 영화 5편 등록, 포스터·평균 감성 점수 표시

영화 추가 화면

제목·개봉일·감독·장르·포스터 URL을 입력해 새 영화를 등록하는 폼입니다. 제목은 필수 입력이며, 비어 있으면 백엔드에서 422 오류로 걸려집니다.

The screenshot shows a web application interface for adding a movie. The left sidebar contains navigation links: '연결 정보' (Connection Info), '백엔드 연결됨' (Backend Connected), '화면 설정' (Screen Settings), and '모든 데이터는 백엔드에서 관리됩니다.' (All data is managed on the backend). The main content area is titled '영화 리뷰 & 감성 분석 서비스' and includes a subtitle 'FastAPI 백엔드 + Streamlit 프론트엔드 · 리뷰 감성 자동 분석'. Below this, there are tabs for '영화 목록' (Movie List), '영화 추가' (Add Movie), and '리뷰' (Review). The '영화 추가' tab is active, showing a form titled '새 영화 등록' (New Movie Registration). The form has five input fields: '제목' (Title) with a placeholder '예: 인터스텔라', '개봉일' (Release Date) with a placeholder '예: 2014-11-06', '감독' (Director) with a placeholder '예: 크리스토퍼 놀란', '장르' (Genre) with a placeholder '예: SF, 드라마', and '포스터 URL' (Poster URL) with a placeholder 'https://... (나무위키 등의 이미지 주소)'. A red '등록하기' (Register) button is located at the bottom of the form. The browser address bar shows 'codeit-movie-review.streamlit.app'.

그림 6-2. 영화 추가 화면 — 영화 정보 입력 폼

리뷰 등록 및 감성 분석 화면

영화를 선택해 작성자와 리뷰 내용을 입력하면, 등록과 동시에 감성 분석이 실행됩니다. 화면 하단에는 **최근 10개 리뷰**가 리뷰 ID·영화 ID·작성자·리뷰 내용·감성 결과·점수와 함께 표시되며, 각 리뷰 행 끝의 삭제 버튼으로 개별 리뷰를 삭제할 수 있습니다. 감성 결과는 긍정·부정·중립으로 직관적으로 구분됩니다.

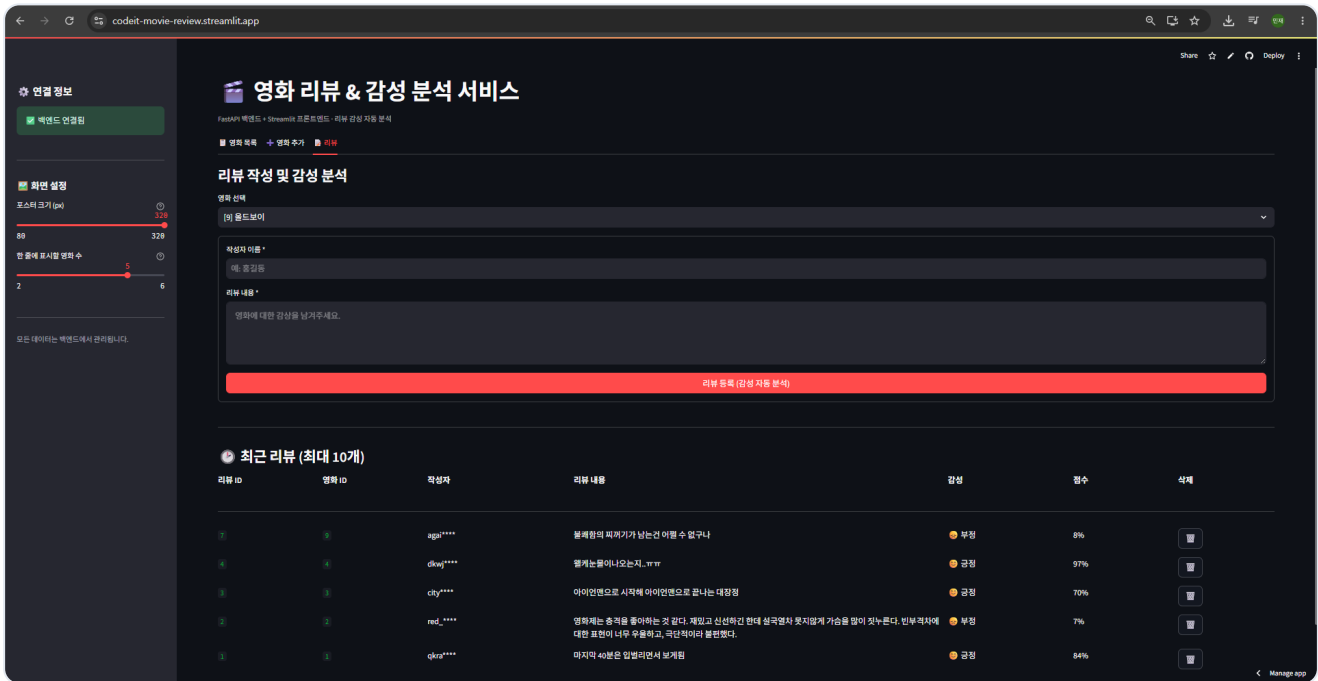


그림 6-3. 리뷰 화면 — 리뷰 작성 폼과 최근 리뷰(감성 분석 결과·삭제 버튼 포함)

7 프로젝트 구조 및 실행 방법

폴더 구조

경로	설명
<code>backend/main.py</code>	FastAPI 진입점 (lifespan, CORS, 라우터 등록)
<code>backend/database.py</code>	sqlite3 기반 연결·테이블 생성·의존성 주입
<code>backend/schemas.py</code>	Pydantic 요청/응답 스키마
<code>backend/sentiment.py</code>	감성 분석 (KcELECTRA 모델 + 규칙 기반 폴백)
<code>backend/auth.py</code>	API 키 인증
<code>backend/routers/movies.py</code>	영화 CRUD 라우터
<code>backend/routers/reviews.py</code>	리뷰 CRUD · 감성분석 · 평점 라우터
<code>backend/Dockerfile</code> , <code>requirements.txt</code>	백엔드 컨테이너 정의
<code>frontend/streamlit_app.py</code>	Streamlit 화면 (영화 목록·추가·리뷰 3개 탭)
<code>frontend/Dockerfile</code> , <code>requirements.txt</code>	프론트엔드 컨테이너 정의
<code>docker-compose.yml</code>	백엔드+프론트엔드 통합 실행

실행 방법

방법 A. Docker Compose (권장)

프로젝트 최상단에서 아래 명령 한 줄로 백엔드와 프론트엔드가 각각 독립된 컨테이너로 실행되며, 서로 통신할 수 있는 네트워크까지 자동으로 구성됩니다. 실행 후 `http://localhost:8501` 에서 화면을, `http://localhost:8000/docs` 에서 API 문서를 확인합니다.

```
docker compose up -d --build
```

방법 B. 로컬 직접 실행

백엔드는 `backend` 폴더에서 `uvicorn main:app --reload` 로, 프론트엔드는 `frontend` 폴더에서 `streamlit run streamlit_app.py` 로 각각 실행합니다. 감성 분석 모델을 사용하지 않고 규칙 기반으로만 빠르게 테스트하려면 환경변수 `USE_ML_MODEL=0` 을 지정하면 됩니다.

배포

본 프로젝트는 실제로 다음과 같이 배포되어 동작합니다. 프론트엔드는 GitHub 저장소와 연동해 **Streamlit Cloud**(`codeit-movie-review.streamlit.app`)에 배포했고, 백엔드는 Docker 이미지를 빌드·푸시하여 **GCP Cloud Run**(`movie-review-api-service...asia-northeast3.run.app`)에 배포했습니다. 배포 시에는 API 키와 백엔드 주소를 코드에 하드코딩하지 않고, Streamlit의 `secrets.toml` 과 백엔드의 환경변수 (`API_KEYS` , `CORS_ORIGINS`)로 관리합니다.

소스 코드와 컨테이너 이미지는 아래에 공개되어 있습니다.

- **GitHub**: https://github.com/minjaejeon0827/sprint_mission18
- **Docker Hub (프론트엔드)**: <https://hub.docker.com/r/minjaejeon0827/frontend-movie-review>
- **Docker Hub (백엔드)**: <https://hub.docker.com/r/minjaejeon0827/backend-movie-review>

마무리 — 본 프로젝트는 화면(Streamlit)과 데이터 처리 로직(FastAPI)을 분리하고, 표준 라이브러리 `sqlite3` 만으로 데이터를 영구 저장하며, 한국어 감성 분석 모델을 경량화 전략과 함께 서빙하는 **실전 적용 가능한 영화 리뷰 감성 분석 서비스**를 완성했습니다.